DEPARTMENT OF TECHNOLOGY

Machine Learning Laboratory

Experiment 04

Multiple Linear Regression using Gradient Descent

Effect of Feature Scaling on Gradient Descent

Learning Objective

To implement Multiple Linear Regression using Gradient Descent and observe the effect of feature scaling on the convergence of the optimization process by comparing the cost across iterations with and without scaling.

1 Aim

To implement Multiple Linear Regression using Gradient Descent on a house-price dataset and compare the convergence behaviour of Gradient Descent with and without feature scaling.

2 Learning Objectives

After completing this experiment, students will be able to:

1. Implement Multiple Linear Regression using Gradient Descent.
2. Calculate predictions using model parameters.
3. Calculate the loss for individual training examples.
4. Calculate the cost function for the complete training dataset.
5. Calculate gradients and update model parameters.
6. Store the cost value after every iteration.
7. Visualize the convergence of Gradient Descent.
8. Apply feature scaling to input features.
9. Compare Gradient Descent with and without feature scaling.
10. Study the effect of learning rate on the convergence behaviour of Gradient Descent.

3 Problem Statement

In the previous experiment, Multiple Linear Regression was implemented using the Scikit-learn library.

In this experiment, the same house-price dataset will be used to implement Multiple Linear Regression manually using Gradient Descent.

The experiment will investigate how the scale of input features affects the optimization process.

The model will be trained under two conditions:

1. Gradient Descent without feature scaling.
2. Gradient Descent with feature scaling.

The cost values obtained during training will be recorded and visualized to compare the convergence behaviour.

4 Dataset Description

The same dataset used in Experiment 03 will be used:

ML_Lab_Multiple_Linear_Regression_House_Prices.csv

The dataset contains 100 observations and the following variables:

Feature	**Description**	**Role**
Area_sqft	Area of house in square feet	Input
Bedrooms	Number of bedrooms	Input
Bathrooms	Number of bathrooms	Input
Age_years	Age of house in years	Input
Distance_to_Metro_km	Distance from metro in km	Input
Parking	Parking availability (0/1)	Input
Floor	Floor number	Input
Balconies	Number of balconies	Input
Price_Lakhs	House price in lakhs	Target

5 Theory

Gradient Descent is an optimization algorithm used to find suitable values of model parameters by iteratively reducing the cost function.

For Multiple Linear Regression, the prediction is:

$$\boxed{\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_p x_p} \tag{1}$$

where:

- $\hat{y}$ = predicted output
- w_0 = intercept or bias
- $w_1, w_2, \ldots, w_p$ = model parameters
- $x_1, x_2, \ldots, x_p$ = input features

5.1 Gradient Descent

Gradient Descent updates the model parameters in the opposite direction of the gradient of the cost function.

The general parameter update is:

$$\boxed{w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}} \tag{2}$$

where:

- w_j = model parameter
- α = learning rate
- J = cost function
- $\frac{\partial J}{\partial w_j}$ = gradient of the cost with respect to parameter w_j

5.2 Cost Function

For Mean Squared Error based Gradient Descent, the cost function is:

$$\boxed{J(w) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2} \tag{3}$$

where:

- m = number of training examples
- y_i = actual value
- $\hat{y}_i$ = predicted value

The cost value is calculated after every iteration and stored to observe the convergence of Gradient Descent.

5.3 Individual Loss

For an individual training example, the squared loss is:

$$\boxed{L_i = \frac{1}{2}(y_i - \hat{y}_i)^2} \tag{4}$$

The overall cost is obtained by averaging the individual losses:

$$\boxed{J = \frac{1}{m} \sum_{i=1}^{m} L_i} \tag{5}$$

Therefore:

$$\boxed{\text{Individual Losses} \rightarrow \text{Cost}}$$

5.4 Gradient for Multiple Linear Regression

For the model:

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_p x_p$$

the gradient with respect to a parameter w_j is:

$$\boxed{\frac{\partial J}{\partial w_j} = -\frac{1}{m}\sum_{i=1}^{m} x_{ij}(y_i - \hat{y}_i)} \tag{6}$$

For the intercept:

$$\boxed{\frac{\partial J}{\partial w_0} = -\frac{1}{m}\sum_{i=1}^{m} (y_i - \hat{y}_i)} \tag{7}$$

Students have already studied the mathematical derivation of these gradients in the theory component.

6 Feature Scaling

The features in the house-price dataset have different numerical ranges.

For example:

$$\text{Area_sqft}$$

may contain values in the hundreds or thousands, while:

$$\text{Bedrooms}$$

may contain values such as:

$$1, 2, 3, 4, 5$$

Therefore, the input features do not have comparable scales.

Feature scaling transforms the input features into a more comparable numerical range.

6.1 Standardization

In this experiment, Standardization will be used.

The standardized feature is calculated as:

$$\boxed{x_{\text{scaled}} = \frac{x - \mu}{\sigma}} \tag{8}$$

where:

- μ = mean of the feature
- σ = standard deviation of the feature

Only the input features are scaled in this experiment.

Important

The target variable `Price_Lakhs` is not scaled for this experiment.
The purpose is to observe how changing the scale of the input features affects Gradient Descent.

7 Experiment Workflow

The experiment follows the workflow:



8 Implementation

8.1 Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
```

8.2 Step 2: Load the Dataset

Load the same dataset used in Experiment 03.

```
df = pd.read_csv(
    "ML_Lab_Multiple_Linear_Regression_House_Prices.csv"
)

print(df.head())
```

8.3 Step 3: Select Features and Target

Use the same eight input features from Experiment 03.

```
features = [
    'Area_sqft',
    'Bedrooms',
    'Bathrooms',
    'Age_years',
    'Distance_to_Metro_km',
    'Parking',
    'Floor',
    'Balconies'
]

X = df[features].values

y = df['Price_Lakhs'].values
```

8.4 Step 4: Inspect Feature Ranges

Before applying scaling, inspect the numerical range of each feature.

```
print(df[features].describe())
```

Observe that the input features may have significantly different numerical ranges.

9 Experiment 1: Gradient Descent Without Scaling

In the first experiment, use the original feature values.

$$\boxed{X = X_{\text{original}}}$$

No scaling is applied.

9.1 Step 5: Initialize Parameters

Initialize the model parameters with zeros.

```
m = len(y)

weights = np.zeros(X.shape[1])

bias = 0

learning_rate = 0.0000001

iterations = 1000
```

The number of weights is equal to the number of input features.
Since there are eight input features:

$$\boxed{w_1, w_2, \ldots, w_8}$$

are initialized.

9.2 Step 6: Implement Gradient Descent

Implement Gradient Descent using the following procedure.

```
cost_history = []

for i in range(iterations):

    # Prediction
    y_pred = X @ weights + bias

    # Error
    error = y_pred - y

    # Individual loss
    individual_loss = 0.5 * (error ** 2)

    # Cost
    cost = np.mean(individual_loss)

    # Store cost
    cost_history.append(cost)

    # Gradients
    dw = (1/m) * (X.T @ error)

    db = (1/m) * np.sum(error)

    # Parameter update
    weights = weights - learning_rate * dw

    bias = bias - learning_rate * db
```

9.3 Step 7: Display Final Cost

```
print("Final Cost:", cost_history[-1])
```

The cost value should be monitored during training.

9.4 Step 8: Plot Cost vs Iteration

```
plt.plot(cost_history)

plt.xlabel("Iteration")
plt.ylabel("Cost")

plt.title(
    "Gradient Descent Without Feature Scaling"
)

plt.show()
```

Observe the behaviour of the cost as the number of iterations increases.

10 Experiment 2: Gradient Descent With Scaling

In the second experiment, standardize the input features before applying Gradient Descent.

$$\boxed{X \rightarrow X_{\text{scaled}}}$$

10.1 Step 9: Apply Standardization

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print(X_scaled[:5])
```

Observe how the numerical values of the features have changed after standardization.

10.2 Step 10: Reinitialize Parameters

The parameters must be initialized again before starting the second experiment.

```
weights = np.zeros(X_scaled.shape[1])

bias = 0
```

Use the same learning rate and number of iterations used in Experiment 1.

```
learning_rate = 0.01

iterations = 1000
```

10.3 Step 11: Implement Gradient Descent on Scaled Data

```
cost_history_scaled = []

for i in range(iterations):

    # Prediction
    y_pred = X_scaled @ weights + bias

    # Error
    error = y_pred - y

    # Individual loss
    individual_loss = 0.5 * (error ** 2)

    # Cost
    cost = np.mean(individual_loss)

    # Store cost
    cost_history_scaled.append(cost)

    # Gradients
    dw = (1/m) * (X_scaled.T @ error)
```

```
    db = (1/m) * np.sum(error)

    # Parameter update
    weights = weights - learning_rate * dw

    bias = bias - learning_rate * db
```

10.4 Step 12: Plot Cost vs Iteration

```
plt.plot(cost_history_scaled)

plt.xlabel("Iteration")
plt.ylabel("Cost")

plt.title(
    "Gradient Descent With Feature Scaling"
)

plt.show()
```

11 Comparison of Scaling and Convergence

Compare the convergence behaviour of Gradient Descent with and without feature scaling.

```
plt.plot(
    cost_history,
    label="Without Scaling"
)

plt.plot(
    cost_history_scaled,
    label="With Scaling"
)

plt.xlabel("Iteration")
plt.ylabel("Cost")

plt.title(
    "Effect of Feature Scaling on Gradient Descent"
)

plt.legend()

plt.show()
```

The graph should be used to compare:

- Rate of decrease of cost
- Stability of the optimization

- Number of iterations required for convergence
- Behaviour of the cost during training

12 Experiment 3: Effect of Learning Rate

After observing the effect of feature scaling, investigate the effect of the learning rate using the scaled dataset.

Try different learning rates such as:

$$\boxed{\alpha = 0.001, \quad 0.01, \quad 0.1}$$

For each learning rate:

1. Initialize the parameters.
2. Run Gradient Descent.
3. Store the cost after every iteration.
4. Plot the cost against iteration.

12.1 Comparison

Compare the convergence behaviour for different learning rates.

Learning Rate	**Observed Behaviour**	**Convergence**
0.001		
0.01		
0.1		

13 Understanding the Cost Curve

The cost value is recorded after every iteration:

$$J_1, J_2, J_3, \ldots, J_T$$

where T is the total number of iterations.

These values are plotted against the iteration number.

An ideal convergence behaviour generally shows a decreasing overall trend:

$$\boxed{J_1 > J_2 > J_3 > \cdots}$$

However, in practical machine learning problems, the cost does not necessarily have to decrease at every individual iteration.

Small fluctuations may occur depending on the dataset, optimization method and learning rate.

A continuously increasing or diverging cost may indicate an unsuitable learning rate or another implementation problem.

14 Observations

Record the following observations:

Measure	Observation
Number of observations	
Number of features	
Initial cost without scaling	
Final cost without scaling	
Initial cost with scaling	
Final cost with scaling	
Convergence without scaling	
Convergence with scaling	
Best learning rate	

15 Result

Multiple Linear Regression was successfully implemented using Gradient Descent.

The individual losses and overall cost were calculated during training.

The cost history was visualized to observe the convergence of Gradient Descent.

The experiment also demonstrated the effect of feature scaling on Gradient Descent and showed how the learning rate influences the convergence behaviour.

16 Post-Lab Questions

1. What is the difference between loss and cost?
2. Why is the cost recorded after every iteration?
3. What is the purpose of the learning rate in Gradient Descent?
4. What happens if the learning rate is too small?
5. What may happen if the learning rate is too large?
6. Why can the cost fluctuate between iterations?
7. Why do the input features have different numerical scales in the house-price dataset?
8. What is the purpose of feature scaling in Gradient Descent?
9. What is Standardization?
10. Why are the model parameters reinitialized before the scaled experiment?
11. Why is the target variable not scaled in this experiment?
12. Compare the convergence behaviour of Gradient Descent with and without feature scaling.
13. Does feature scaling change the number of features in the dataset?
14. Does feature scaling change the underlying Gradient Descent algorithm?

17 Additional Tasks

1. Try a different number of iterations and observe the cost curve.
2. Experiment with at least three different learning rates.
3. Compare the convergence behaviour for scaled and unscaled features.
4. Identify a learning rate that gives stable convergence for the scaled dataset.
5. Plot the final cost curves for the best learning rate with and without scaling.

18 Important Note

Important

In this experiment, the objective is to understand the implementation and behaviour of Gradient Descent.
Students should focus on:

$$\boxed{\text{Prediction} \rightarrow \text{Individual Loss} \rightarrow \text{Cost} \rightarrow \text{Gradient} \rightarrow \text{Parameter Update}}$$

and then compare:

$$\boxed{\text{Without Scaling} \quad \text{vs.} \quad \text{With Scaling}}$$

Feature scaling is particularly important for optimization methods such as Gradient Descent because the numerical scale of input features can affect the size and direction of parameter updates.

Key Learning Flow

$$\boxed{\begin{aligned}\text{House Dataset} &\rightarrow \text{MLR Model}\\ &\rightarrow \text{Prediction}\\ &\rightarrow \text{Loss}\\ &\rightarrow \text{Cost}\\ &\rightarrow \text{Gradient}\\ &\rightarrow \text{Parameter Update}\\ &\rightarrow \text{Cost History}\\ &\rightarrow \text{Convergence Curve}\\ &\rightarrow \text{Feature Scaling}\\ &\rightarrow \text{Compare Convergence}\end{aligned}}$$